Automatic Road Damage Detection from Street Images using Deep Learning

Project Defence

Muhammad Sultan Haidar Cheema

International Master in Machine Learning and Data Mining (MLDM)
Université Jean Monnet Saint-Étienne

August 2026

Outline

1. Introduction
2. Related Work
3. Dataset & Methodology
4. Results
5. Cross-Country Generalization
6. Deployment & Conclusion

Motivation & Problem

- Manual road inspection is slow, labour-intensive, and often subjective – yet essential for road safety.
- Deep learning, especially the YOLO family, can detect damage automatically and in real time.
- **The gap:** prior studies rarely use identical settings for comparison, and almost never test whether a model still works in a different country.

This project

A fair, apples-to-apples comparison of three YOLO models – plus a real test of cross-country generalization.

Prior Approaches: Traditional & Machine Learning

- **Thresholding** – convert to grayscale; cracks are darker than the pavement. Otsu's method automatically picks the intensity threshold from the image histogram.
- **Edge detection** – Sobel estimates the intensity gradient to highlight boundaries; Canny refines this with noise smoothing, thinning, and dual thresholding for cleaner edges.
- **Classical ML** – handcrafted features (HOG, LBP) fed into an SVM or Random Forest classifier.

Shared limitation

All three are easily confused by shadows, stains, and lighting changes, and require manual tuning for each scene.

Prior Approaches: Deep Learning

- A plain CNN classifier tells you **what** is in an image, not **where**.
- Region-based detectors (R-CNN family) add localization: first **propose** candidate regions, then run a CNN classifier on **each region separately**.
- Running the CNN hundreds of times per image is accurate, but **computationally expensive**.

The bottleneck

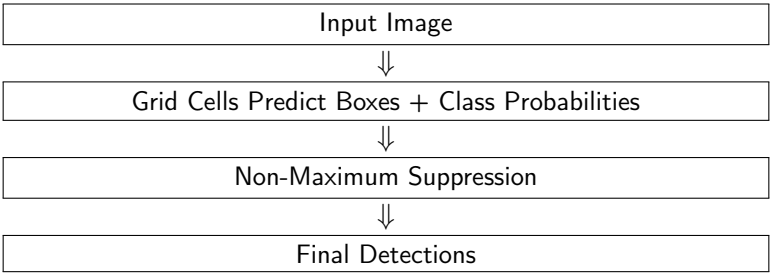
Too slow for real-time road inspection from a moving vehicle – this motivated one-stage detectors such as YOLO, which skip the separate proposal step entirely.

Two-Stage vs. One-Stage Detectors

	Two-Stage	**One-Stage**
Examples	R-CNN, Fast R-CNN, Faster R-CNN	SSD, RetinaNet, **YOLO**
Approach	Propose regions, then classify each	Predict boxes + classes in a single pass
Speed	Slower (multi-stage pipeline)	Fast, real-time capable
Best for	Offline, high-precision analysis	Live inspection from a moving vehicle

How YOLO Works

- Divide the input image into an $S \times S$ grid.
- Each cell directly predicts bounding box coordinates, an objectness score, and class probabilities – all in **one forward pass**.
- **Non-Maximum Suppression (NMS)** removes duplicate overlapping boxes, keeping only the highest-confidence detection per object.



Localization and classification happen together → "You Only Look Once."

Evolution of the YOLO Family

Version	Year	Key Improvement
YOLOv1	2016	Single-stage detection in one forward pass
YOLOv2	2017	Anchor boxes, batch normalization, multi-scale training
YOLOv3	2018	Darknet-53 backbone, multi-scale prediction
YOLOv4	2020	CSPDarknet53, Mosaic augmentation, PAN
YOLOv5	2021	PyTorch implementation, simplified deployment
YOLOv6	2022	Optimized for industrial deployment
YOLOv7	2023	Trainable Bag-of-Freebies, re-parameterization
YOLOv8	2023	Anchor-free head, unified multi-task framework

This project uses YOLOv5s, YOLOv8n, and YOLOv8s (highlighted above).

Objectives

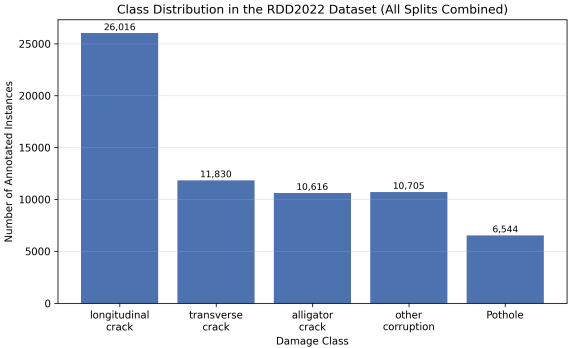
- Train and compare YOLOv5s, YOLOv8n, YOLOv8s (Precision, Recall, mAP, FPS).
- Analyse detection performance per damage category.
- Evaluate cross-country generalization: train on Japan, test on six countries.
- Deploy the best model in an interactive Streamlit application.

RDD2022 Dataset

- Road images from **six countries**: Japan, India, Czech Republic, Norway, China, United States.
- **5 damage classes**: longitudinal crack, transverse crack, alligator crack, other corruption, pothole.

Split

Train	26,869 images
Validation	5,758 images
Test	5,758 images



Training Setup

Three YOLO models compared:

- **YOLOv5s** – lightweight baseline
- **YOLOv8n** – smallest of the latest generation
- **YOLOv8s** – larger, higher-accuracy variant

Identical Settings

- Image size: 640×640
- Batch size: 16, Epochs: 50
- Patience: 15
- GPU: NVIDIA Tesla T4

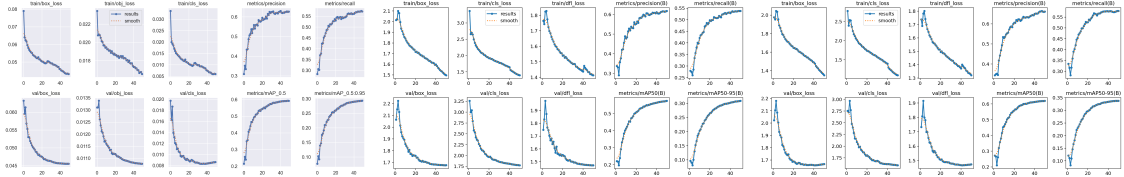
Fair comparison

Same dataset, same settings – differences in results come from the model architecture.

Experimental Flow

1. **Train & compare** YOLOv5s, YOLOv8n, YOLOv8s under identical settings; evaluate on the RDD2022 test set.
2. **Per-class analysis** – break down performance by damage category.
3. **Cross-country generalization** – retrain the best model on Japan only, test on all six countries.
4. **Deployment** – integrate the best model into a Streamlit application.

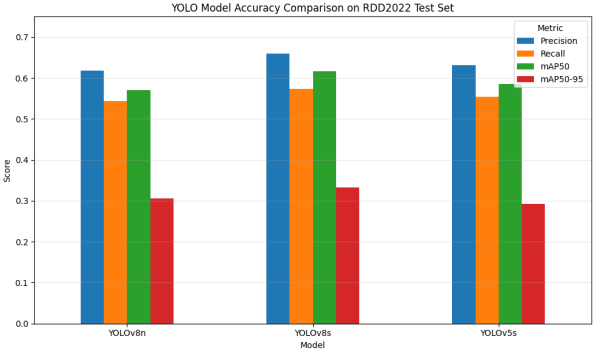
Training Behaviour



YOLOv5s YOLOv8n YOLOv8s

All three models converge smoothly; YOLOv8s reaches the highest validation performance.

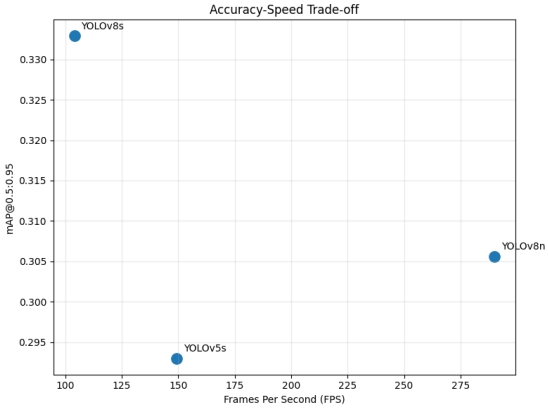
Test-Set Comparison

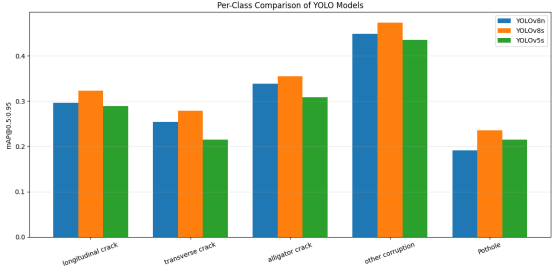


Key result

YOLOv8s: best accuracy (mAP@0.5 = 0.6170). YOLOv8n: fastest (290 FPS, nearly 3× YOLOv8s).

Speed–Accuracy Trade-off & Per-Class Performance





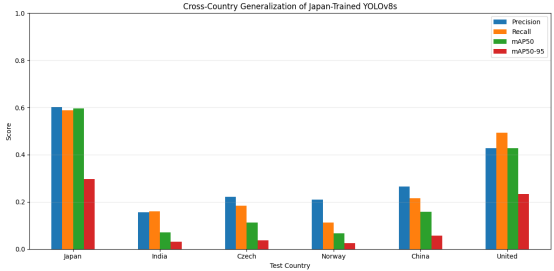
Larger models trade speed for accuracy; **Other Corruption** is easiest to detect, **Pothole** the hardest (rarest class in training data).

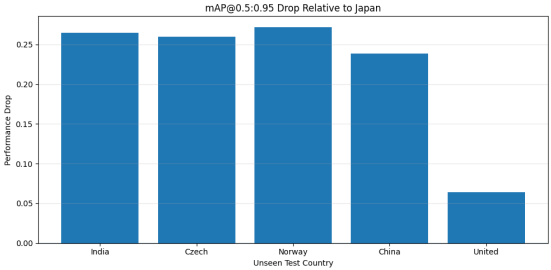
Cross-Country Generalization

- YOLOv8s retrained using **only Japanese** images, tested on all six countries.

Country	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Japan (seen)	**0.6012**	**0.5883**	**0.5966**	**0.2965**
United States	0.4282	0.4927	0.4282	0.2327
China	0.2656	0.2161	0.1581	0.0576
Czech Republic	0.2208	0.1840	0.1122	0.0363
Norway	0.2097	0.1129	0.0673	0.0244
India	0.1557	0.1593	0.0711	0.0315

Cross-Country Charts



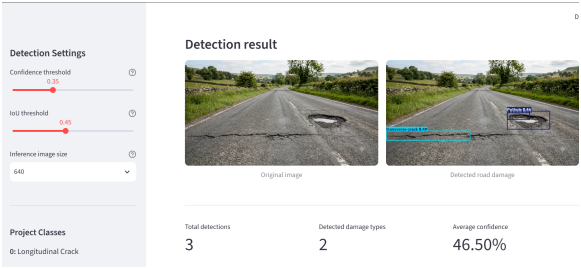


Key finding

Performance drops sharply on unseen countries – pavement, weather, and camera differences cause real domain shift.

Streamlit Application

- Upload a road image, adjust confidence/IoU thresholds.
- Displays annotated image, detection table, and summary.
- Results exportable as CSV.



Conclusion & Future Work

- **YOLOv8s**: best accuracy; **YOLOv8n**: fastest inference.
- Cross-country test reveals significant **domain shift** on unseen countries.
- Working **Streamlit app** demonstrates practical deployment.

Future Work

More geographically diverse training data, domain adaptation, and edge-device optimization.

Thank You

Questions?